



CONVERGE

THE COMPLETE DOCUMENTATION · v0.1.0

Compile intent into shipped software.

DONE IS PROVEN. – never claimed by the agent.

The dark factory for coding agents: a raw idea enters a **nine-gate production line** and leaves as merged, proven work. Autonomous agents do the building; machines do the checking. One method, one referee CLI, twelve portable skills – **zero runtime dependencies**.

CLAUDE CODE

CODEX

KIMI

GROK BUILD

BASH 3.2+

MIT

Contents. Nine chapters, one production line.

CHAPTER 01 Introduction What is Converge?03 The vocabulary04	CHAPTER 02 The method The descent – nine passes05 Gates, tokens & evidence06	CHAPTER 03 The passes, one by one 0 Capture · 1 Intent · 2 Structure07–09 3 Decompose · 4 Consensus10–11 5 Tasking + the six-tier engine12–13 6 Register · 7 Bind · 8 The Loop14–16 The loop state machine17
CHAPTER 04 The trust chain Seals, tiers & receipts18 Proof in production19	CHAPTER 05 The skills Utilities & the twelve-skill catalog20	CHAPTER 06 The referee CLI The surface at a glance21 The agent protocol22
CHAPTER 07 Architecture Control plane & the write fence23 Cockpit – read-only observability24	CHAPTER 08 Operating the factory Steering from chat25	CHAPTER 09 Install & status Three doors, one result26 Status, provenance & roadmap27

How to read this document: every pass chapter follows one template – identity → job → gate → CLI path → artifact anatomy → failure modes. Every claim ends in a greppable token you can verify in your own repository.

What is Converge? The factory where done is proven.

Converge **compiles intent into software**. Whatever coding agent you run – Claude Code, Codex, Kimi, Grok Build – it works against **signed task specs whose completion is enforced by evals**, not by the agent's word. Every task ships with runnable bash evals authored **before** the work; a task only settles when those evals exit green.

The unit of work is the **Task-Spec**: a self-verifying markdown file with a machine-checked format, an effort budget, an HMAC sign-off seal, and its own definition of done. The **cvg** CLI is the **referee** around it – it frames, dispatches, and gates, but holds zero model credentials and never writes a line of product code.

"Agents play; the referee scores."

PLAIN BASH + STDLIB PYTHON – THE SAME INSTALL SERVES EVERY HARNESS

WHY · 01

The eval decides done

A task cannot settle until its evals exit 0 – completion is a state-machine invariant. The maximal stub attack (all nine specs faked) lands all nine RED.

WHY · 02

The referee is never a player

cvg holds no API keys and calls no models. Engine CLIs authenticate themselves; the referee only frames, dispatches headlessly, and gates.

WHY · 03

Cross-family verification

Tier-2 acceptance sends the diff, the intent, and holdout criteria the builder never saw to a *different vendor's* model, prompted to refute.

WHY · 04

A loop with brakes

Attempt → verify → repeat under three-axis budgets, a stagnation detector, and eight named terminal states. An exhausted budget is never a success.

9

SEQUENTIAL GATES

12

PORTABLE SKILLS

4

HARNESSES · ONE METHOD

0

MODEL CREDENTIALS HELD

52

HERMETIC LOOP CHECKS

3

LANES · FAST NORMAL FULL

Twelve words that carry the whole system.

THE SHAPE

The descent

The nine-gate production line, pass 0 → 8. A raw idea goes in; merged, proven work comes out. Evidence and lessons flow back to feed the next pass.

THE UNIT OF METHOD

Pass

One installable skill with **one job, one output, one gate**. You or your agent do the work; the gate proves it happened.

THE VERDICT

Gate & token

A deterministic script whose verdict ends in one greppable token (`CHECK_PLAN=OK`). Fail-closed: a verdict that cannot be read is never a pass.

THE UNIT OF WORK

Task-Spec

A self-verifying markdown file: machine-checked format, effort budget, HMAC seal, and its own definition of done. The evals travel **inside** the file.

THE JUDGE

Eval

Runnable bash authored **before** the work. The eval decides done — completion is a state-machine invariant, never an agent's claim.

THE TRUST DEVICE

The seal

An HMAC-SHA256 envelope over the spec's identity, body digest, and authorization fields. Edit one eval byte after signing and the gate refuses.

PASS 4

The barrier

A different-family model attacks the plan; the owner resolves objections and signs. **The last human decision in the chain.**

PASS 5

The cornerstone

Where intent becomes signed, self-verifying Task-Specs — the point of no return between the design half and the machine half.

THE ROUTER

Lane

`cvg lane` routes work to the ceremony it earns — FAST (5, 7, 8) · NORMAL (1, 2, 5, 7, 8) · FULL (0–8). It routes, but **never waives**.

THE UMPIRE

The referee

`cvg` — frames, dispatches, gates. Holds zero model credentials, calls no models, writes no product code. Agents play; the referee scores.

THE TERMINAL STATE

Settled

Green evals within budget, receipt written, PR opened — `TASK_LOOP=SETTLED`. An exhausted budget is never a success.

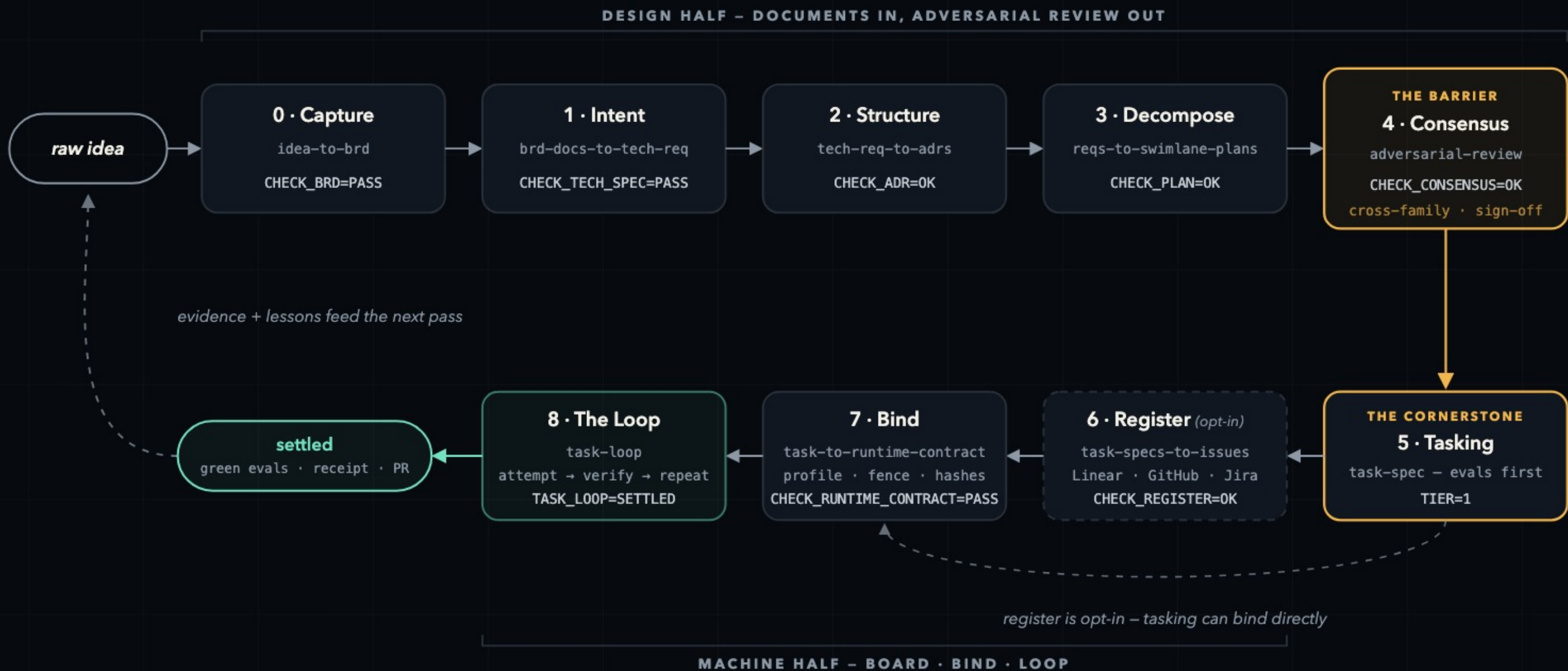
THE FLOOR

Evidence

Receipts, seals, tokens, objection logs, attempt transcripts — the artifacts every gate stands on. `cvg next` derives the whole descent's position from them.

The three layers: the **Agentic Loop** is the runtime (attempt → verify → repeat) · **Converge** is the method (nine gated passes) · **ASD** — Agentic Software Delivery — is the meta-process that turns every run's evidence into the next run's input.

The descent. One job, one output, one gate – per pass.



LANE ROUTING – cvg lane routes work to the ceremony it deserves. It routes, but never waives: nothing irreversible rides FAST, and no lane dispatches an unsigned spec.

- FAST** a contained change – task, bind, loop
- NORMAL** real features – requirements and structure first
- FULL** new systems – the whole descent, barrier included



How a gate works. Fail-closed, byte-exact, greppable.

THE MECHANICS

Every wrapped gate is a **byte-exact pass-through** (exec) to the proven skill script – the router adds convenience, never behavior. Each verdict ends in one token on its own line, so a script, a CI job, or an agent can read the outcome with a single grep. Artifacts are discovered in typed folders (cvg/docs/brd/, then legacy paths) – zero found is exit 2, never a silent pass.

FAIL-CLOSED, EVERYWHERE

A verdict that cannot be obtained is never a pass: a missing engine is **SKIP**, a wall-clock breach is **TIMEOUT**, an unparseable judgment is **ENGINE_ERROR**. Gate checks run with **fenced code blocks stripped**, so pasting the magic words inside a code sample authorizes nothing. Sentinels like (open), tbd, pending read as **unresolved** – anything the script cannot prove resolved counts against you.

THE CONDUCTOR – cvg next

Derives which pass is current and which comes next **from workspace evidence alone** – no stored state. It prints the evidence board, the owning skill, its steering prompt, and the pass's closing gate. Presence probes only: evidence presence is not a verdict – the gates stay authoritative. Pre/post hooks enforce descent order fail-closed. **NEXT_PASS=0-8|DONE**

THE TOKEN LEDGER – ONE VERDICT PER PASS

0	Capture	CHECK_BRD=PASS FAIL USAGE_ERROR
1	Intent	CHECK_TECH_SPEC=PASS FAIL USAGE_ERROR
2	Structure	CHECK_ADR=OK FAIL EMPTY USAGE_ERROR
3	Decompose	CHECK_PLAN=OK FAIL EMPTY USAGE_ERROR
4	Consensus	CHECK_CONSENSUS=OK FAIL EMPTY USAGE_ERROR
5	Tasking	VERDICT: DELEGATE · TIER=1 2 3
6	Register	CHECK_REGISTER=OK FAIL
7	Bind	CHECK_RUNTIME_CONTRACT=PASS FAIL
8	The Loop	TASK_LOOP=SETTLED ...(8 states)
·	Tier-2	CHECK_VERIFY=UPHELD REFUTED UNAVAILABLE ERROR

EVIDENCE HAS ITS OWN DOCTOR

```
$ cvg doctor evidence      # can git show the floor the gates stand on?
$ cvg doctor               # ≥2 engines, ≥1 cross-family → DOCTOR=OK
$ cvg doctor plugin        # is this project loading a STALE Converge?
```

— **Why tokens:** a token is a contract between the gate and everything downstream – scripts, CI, agents, and this document. If a claim in these pages matters, it names the token that proves it.

0 Capture – grill the gaps out of the idea.

SKILL · idea-to-brd

GATE · CHECK_BRD=PASS

OPTIONAL · FULL LANE

IN · A RAW IDEA

OUT · cvg/docs/brd/<slug>.md

WHAT YOU DO

The skill grills the **stakeholder**, not the engineer, in **frontier rounds** – every answerable question goes out at once, numbered, each with a recommended default. Seven branches in leverage order: pain → **the do-nothing test** → goal → scope boundary → success from the owner's seat → constraints → the pre-mortem (*"it shipped and failed – what killed it?"*).

WHAT THE GATE PROVES

The brief is complete and signed: verdict line **canonical** in the owner's hand, a calendar-real date, every number provenance-tagged, every open question owned. Checks run with **fenced code blocks stripped** – evasion by example is blocked.

THE CLI PATH

- 1 cvg capture the BRD exit contract
- 2 cvg capture --draft progress check – never authorizes
- 3 cvg capture --no-go validate the parked record

CHECK_BRD=FAIL

A (guessed) number with no linked open question fails. A scope entry that says nothing fails. A heading that drifts from the template fails. Owner-voice is a WARN; structure is law.

cvg/docs/brd/storm-alerts.md

ANATOMY OF A BRD

```
## Executive summary – written LAST, ≤ 60 words
## Problem – every number tagged (measured) / (estimated) / (guessed)
## Goals & KPIs – in the owner's terms, not the engineer's
## Scope – in / out, each entry earns its line
## Definition of success · ## Stakeholders · ## Risks · ## Constraints
## Open questions – every one carries a named owner
## Sign-off verdict: canonical date: 2026-08-04 – born _pending_, only the owner flips it
```

THE DO-NOTHING EXIT

cvg/docs/no-go/<slug>.md

If the do-nothing cost is tolerable: the idea in two lines, the owner of the call, the do-nothing answer **verbatim**, and what would reopen it. Parked, not deleted – and no handoff from a no-go.

ANSWER DISCIPLINE

Locked: 1 – <answer>

Answers are locked by restating. Push back once per vague answer; after two carries an unanswered question becomes an **owned open question**, never a silent gap.

1 Intent – every requirement falsifiable.

SKILL · brd-docs-to-tech-req

GATE · CHECK_TECH_SPEC=PASS

LANES · NORMAL + FULL

IN · THE BRD

OUT · cvg/docs/tech-spec/<slug>.md

WHAT YOU DO

Read the BRD as the senior engineer accountable for delivery. The admission rule: a requirement enters the spec **only if a future eval could pass or fail it**. Non-falsifiable wishes go to the open-assumptions list with a named owner – never softened into the spec.

WHAT THE GATE PROVES

Verdict **canonical** with a valid date, and **no blocker gap unresolved**. The blocker check fails CLOSED: (open), tbd, pending – quoted or not – all count as unresolved.

THE CLI PATH

- 1

cvg intent --draft

progress check while drafting
- 2

cvg intent

the Pass 1 exit contract

ALTITUDE GUARD

Stack names leak downward: a tech choice in the spec is Pass 2's altitude and draws a WARN, bucketed by domain. An R-n with no number, unit, or comparator draws another.

● ● ● cvg/docs/tech-spec/storm-alerts.md

ANATOMY OF A TECH-SPEC

```
## TL;DR – ≤ 25 words; the outcome in ≤ 3 sentences
## Requirements – stable ids, never renumbered
R-1 alert latency p95 < 60s from event to publish
R-2 feed availability ≥ 99.5% monthly
W-1 (wish – explicitly out of `must`) per-user quiet hours
## Success metrics – current → target, traced to BRD KPIs
## Gap register – typed, owned, fail-closed on blockers
id: GAP-001 · type: data · severity: blocker · owner: luan · resolution: resolved 2026-08-02 –
source confirmed
```

THE TRACEABILITY CHAIN STARTS HERE

R-n → ADR (spec_ref) → plan → eval
R-n / W-n are what ADRs, plans, and evals cite for the rest of the descent – which is why they are never renumbered.

DECISIONS LIVE ON DISK

Confirmed decisions recap
A **Locked**: decision is written into the spec's decision record the moment it locks; drafting opens with a recap replay – nothing decided “in the air”.

2 Structure – record what is, as ADRs.

SKILL · tech-req-to-adrs

GATE · CHECK_ADR=OK

LANES · NORMAL + FULL

IN · TECH-SPEC + THE REAL REPO

OUT · docs/adrs/ + CONTEXT.md

WHAT YOU DO

Run the real system, don't read about it. Reconcile the spec against the terrain and record each grounding fact as a numbered ADR. Three-condition worthiness test: hard to reverse · downstream passes stand on it · it could plausibly have been otherwise. Fails the test → a glossary line, not a numbered file.

WHAT THE GATE PROVES

The decision set is canonical and consistent: `0000-context.md` present, every section filled, supersede links bidirectional – and under `--final`, nothing still proposed.

THE CLI PATH

- 1 `scaffold-adrs.sh "<slug>"` auto-numbered, dated scaffold
- 2 `scaffold-adrs.sh --supersede NNNN` replace, never edit
- 3 `cvq structure` the ADR gate
- 4 `cvq structure --final` nothing left proposed

NO-DRIFT

A **build-verb** in any ADR fails the gate – build, create, implement, add. ADRs record what **is**, never what to build. That single invariant is this pass's one failure mode.

docs/adrs/0003-source-ships-wal-replica.md

ANATOMY OF AN ADR

```
status: accepted # proposed → accepted → deprecated | superseded
ground: brownfield · spec_ref: R-2 · supersedes: –

## Context · ## Decision · ## Rejected reading
## Evidence – the observation AND the runnable command that re-proves it
$ docker exec pg psql -c 'show wal_level' → replica
## Consequences – always ends with "Re-verify when: ..."
```

ACCEPTED = IMMUTABLE

--supersede NNNN

An accepted ADR is never edited – it is superseded, stamping **supersedes:** / **superseded_by:** on both files. The gate verifies the links point at each other.

THE VOCABULARY LEDGER

docs/CONTEXT.md

Domain terms crystallize as you ground – canonical name, one-line meaning, the evidence line each traces to. Written as it happens, never batched.

3 Decompose – cut the system at its real seams.

SKILL · reqs-to-swimlane-plans

GATE · CHECK_PLAN=OK

LANES · FULL

IN · ADRs + IN-SESSION UNDERSTANDING

OUT · swimlanes/<seam>/

WHAT YOU DO

The decomposition chain: **seam** → **swimlane** → **leg** → (task-spec, at Pass 5). A seam is the joint; a lane gets one lean PRD; a **leg** is the lane's named stretch – one responsibility, one proving test, yielding 1:N task-specs later. Plan altitude only: no query bodies, no handler bodies, no evals.

FIVE HARDENING RULES

H1 exactly one steel-thread lane (**thread=yes** – the walking skeleton) · **H2** spike the riskiest seam first · **H3** one owner per seam (Conway) · **H4** seam evolution declared per consumer · **H5** no unbroken dependency cycles.

THE CLI PATH

- 1 `new-plan.sh "<seam>"` scaffold the lane PRD
- 2 `new-plan.sh --lane <seam> --leg NN-<tech>` scaffold a leg
- 3 `cvlg decompose` tree shape + dependencies

HUMILITY CLAUSE

A green check proves legs **exist, are named, and are ordered** – whether they are well-cut is Pass 4's judgment, not a grep's.

swimlanes/alerts-pipeline/_lane.md · leg-02-ingest.md

ANATOMY OF A LANE + LEG

```
lane-meta: thread=yes · risk=med · owner=data-platform
## Seam · ## Architecture – flowchart + numbered steps (dual coding)
## Non-Goals – the #1 anti-bloat device · ## Legs index
— leg file —
leg: swimlane-alerts-pipeline-leg-02 # the stable key; <tech> is swappable display
depends_on: [...leg-01] · status: accepted
## Responsibility – one job · ## Proves – Given/When/Then, never an eval
## Yields – the 1:N task-specs; read directly by cvlg tasks plan
```

THE DEEP-LANE TEST

interface answers everything

A lane must be answerable from its interface alone – what it does, how you use it, what it depends on, and **what is its secret** (the decision most likely to change).

GATE REFUSALS

CHECK_PLAN=FAIL

No steel thread · non-contiguous legs · an orphan leg or dangling index row · altitude-guard hits · a plan that re-decides an ADR.

4 Consensus – a different family attacks the plan.

SKILL · sketch-plans-adversarial-review

GATE · CHECK_CONSENSUS=OK + OWNER SIGN-OFF

LANES · FULL

IN · SWIMLANE TREE + ADRs

OUT · SHARPENED PLANS + LOG

WHAT HAPPENS

A model from a **different vendor family** (author Claude → adversary Codex, Kimi, or Gemini) attacks the plans **default-to-refuted**: the 5-7 highest-leverage objections, ranked by build-time damage. Hunting grounds: unverified assumptions about existing state · build order · cross-lane interfaces. Empirical disputes get a **throwaway prototype**, not more prose.

THE OBJECTION LIFECYCLE

Every objection ends in exactly one of **FIX** (plan revised in place) or **ACCEPT** (recorded risk + **named owner** + reason to proceed). Nothing is silently dropped – the gate greps one resolution per objection. The plans are sharpened **in place**: the diff is the record; no new files, no new scope.

THE CLI PATH

- 1 `cvg doctor` ≥2 engines, ≥1 cross-family
- 2 `cvg review --adversary codex,kimi` merged, referee-stamped log
- 3 `cvg review --resolve C7 --fix` record the owner's decision
- 4 `cvg review --check` provenance re-hashed

UNSIGNED = STOP

Gate green but nobody signed → **STOP**. Never cut task-specs on an unsigned plan. The owner signs *“this is the right thing to build, and I am handing it to the machine”* – not *“the code will be correct”*. That's the eval's job.

swimlanes/.consensus/objection-log.json

ANATOMY OF THE OBJECTION LOG

```
"provenance": { engine: codex, family: openai, inputs: [sha256..., sha256...] }
// the stamp is the proof – the gate re-hashes the live plans;
// drift since review, or a same-family stamp, fails the gate
"objections": [
  { id: C2, target: swimlane-alerts-pipeline-leg-03, rank: 1,
    finding: "consumer needs event_id – producer never emits it",
    resolution: FIX – leg-02 contract amended },
  { id: C5, resolution: ACCEPT – owner: luan, risk recorded } ]
"fork": { choice: "B" } – frozen compatibility token, echoed atop every PRD
```

WHY CROSS-FAMILY

self-review under-reports

Self-preference bias is family-correlated: a model reviewing its own family's plan blesses too much. A fresh same-family session is the explicitly **weaker** fallback.

SCOPE IS FROZEN HERE

new requirement = drift

A new requirement appearing during review is logged and pushed **back up the chain** – never added as scope. The barrier sharpens; it does not grow.

5 Tasking – the cornerstone.

SKILL · task-spec

GATE TOKEN · TIER=1

LANES · FAST + NORMAL + FULL

UPSTREAM · P4 CONSENSUS

DOWNSTREAM · P6 REGISTER / P7 BIND

WHAT YOU DO

Author Task-Specs from the signed plan: **evals first**, then the effort budget, then the HMAC seal. Each spec is atomic, sized on the six-tier scale, and carries its own definition of done.

WHAT THE GATE PROVES

Each spec is **atomic, sized, and safe to delegate** – the sign-off answers **VERDICT: DELEGATE** and stamps the seal. From that byte on, editing an eval breaks the seal.

THE CLI PATH

- 1 `cvg tasks plan` read-only preview + rationale
- 2 `cvg tasks new <slug> <effort>` scaffold – evals first
- 3 `cvg tasks validate <spec>` structure + sizing gate
- 4 `cvg tasks gate --stamp <spec>` verdict + HMAC seal
- 5 `cvg eval <spec>` run the spec's evals

SEAL=BROKEN If anyone – agent or human – edits an eval after signing, the seal breaks and the gate refuses. Hand-stamping is rejected; the only path to autonomy is through the gate.

ANATOMY OF A TASK-SPEC

cvg/tasks/T-20260730-add-health-endpoint.md

```
1 --- identity – machine-checked frontmatter
id: T-20260730-add-health-endpoint
effort: XS # six-tier scale – sized before it is signed
tier: 1 # deterministic evals decide settlement
---

2 ## Evals – authored before the work
```bash
curl -sf localhost:8080/health | grep -q "ok"
test "$(curl -so /dev/null -w '%{http_code}' localhost:8080/health)" = 200
```

3 ## Budget – three axes, exhaustion is never success
iterations: 5 · wall-clock: 900s · tokens: capped

4 ## Definition of done – traceability back to the plan
green evals · receipt written · PR opened → TASK_LOOP=SETTLED
```

⑤ SIGNED · HMAC-SHA256 · VERDICT: DELEGATE seal breaks if one eval byte changes

GATE A · STRUCTURE

`cvg tasks validate <spec>`
Machine-checked format, six-tier sizing, evals present – atomicity before ceremony.

GATE B · SIGN-OFF

`cvg tasks gate --stamp <spec>`
The verdict + the HMAC seal – stamped at the moment a human says “safe to delegate”.

— **The point of no return:** after this pass, humans stop writing and machines stop trusting – everything downstream runs on the seal, the budgets, and the evals authored here.

The six-tier engine. Size is a budget the gate enforces, not a guess.

Four runnable leaves and two decomposition nodes. Only the **write-surface budget** is gate-enforced; reads and eval counts are authoring guidance. Breach the budget and the verdict is your decomposition was too coarse.

XS

≤ 1 write path
≤ 2 files read · 1 eval
mechanical, tightly specced

S

≤ 2 write paths
≤ 4 files read · 1–2 evals

M

≤ 3 write paths
≤ 6 files read · 2–3 evals

L

≤ 5 write paths
≤ 10 files · ≤ 4 evals
ONE coherent goal

XL

node – children: ≥ 2
never runnable
dispatch the children

XXL

node – children: ≥ 3
never runnable
no route out

TWO SHAPE RULES FOR EVERY ATOM

Cut tracer bullets, not layers – a vertical slice through schema → transform → serving → test beats a horizontal “all the models”. And **size to one fresh context window**: spec + cited ADRs + touched files + working diff, verifying as one PR. The one exception: wide refactors sequence as **expand** → **migrate** → **contract**.

SEVERITY-SCALED CONFIDENCE

The delegation threshold scales with what breaks if the spec is wrong: cosmetic **0.80** · refactor **0.85** · feature **0.90** · bugfix **0.95** (the default) · security **0.98** · financial-critical **0.99**.

STUB-RESISTANCE An eval that only asserts a file exists is satisfied by `echo "# assert" > x.py` – the gate **blocks** existence-only evals. Assert the **outcome**: a row in a table, a role in the cluster, a revoked privilege.

HOLES ARE FIRST-CLASS

An unresolved question makes the spec **status: blocked** with a **blocked_reason:** and a non-empty Open Questions zone – and a blocked atom **never reaches the gate**. Ambiguity is recorded, not smuggled.

BACKLOG-WIDE LINTING

`cvg lint` checks cycles and overlaps across every spec – needs bash 4+ (stock macOS 3.2 prints `LINT=UNSUPPORTED`, exit 3; `brew install bash` and it re-execs itself).

6 Register – the board that cannot lie.

SKILL · task-specs-to-issues

GATE · CHECK_REGISTER=OK

OPT-IN · ALL LANES

IN · SIGNED SPECS

OUT · LINEAR / GITHUB / JIRA ISSUES

WHAT IT DOES

Idempotently projects each `signed_off: true` spec onto **exactly one** tracker issue – never fan out, never merge – and mirrors every `depends_on` edge as the tracker's own **blocked-by** relation. The ready frontier renders natively in the board UI.

THE SIX-VERB ADAPTER

Every tracker is one adapter file speaking six verbs: `preflight` · `upsert` · `link` · `list-ready` · `list-issues` · `write-result`. The loop uses only two; registration uses two; the parity gate reads one. A no-network fake adapter proves the whole suite offline – 145 checks.

THE CLI PATH

- | | | |
|---|---------------------------------------|---------------------------------|
| 1 | <code>cvg setup tracker linear</code> | key → OS keychain, never argv |
| 2 | <code>cvg register --dry-run</code> | detects cycles before any write |
| 3 | <code>cvg register</code> | specs → issues, 1:1 |
| 4 | <code>cvg register --check</code> | parity · DAG · no orphans |

CYCLE = REFUSE ALL A `depends_on` cycle refuses the whole run – it's a spec bug, not a board state. A failed preflight **stops** rather than half-registering.

● ● ● linear issue CVG-23 ≠ cvg/tasks/T-20260730-add-health-endpoint.md

ANATOMY OF THE MAPPING

```
title: the spec's title, verbatim
body: goal + touches_paths + the Exit Check, verbatim – the issue carries its own proof
key: <!-- task-spec: T-20260730-add-health-endpoint --> – the marker IS the idempotency key
deps: blocked-by ≠ depends_on, edge for edge
receipt: tracker_ref: linear:CVG-23 – stamped into frontmatter METADATA;
        the HMAC seal covers id + body + authz – the receipt never breaks it
```

THE ONE RULE THAT KEEPS IT HONEST

only a green eval closes an issue

Only the loop writes result state. Humans and agents can look at the board; **only settlement moves it** – which is why the board never lies.

PARITY, GATED

issues 8 != specs 7 → FAIL

Count mismatch, orphan issue, double-registered spec, or a missing **blocked-by** edge – each one fails **CHECK_REGISTER**.

7 Bind – the smallest enforceable contract.

SKILL · task-to-runtime-contract

GATE · CHECK_RUNTIME_CONTRACT=PASS

LANES · FAST + NORMAL + FULL

IN · ONE SIGNED LEAF SPEC

OUT · cvg/execution/<task-id>/

THE CAPABILITY EPOCH

Authority is granted against **one signed revision**: `epoch = <task-id>@<spec-sha12>`, scoped to the spec's own paths, and **revoked** on settle, block, budget exhaustion, or epoch change – closing the hole where session permissions outlive the task.

PREVENT / DETECT / CANNOT-HONOR

Each runtime adapter declares, per capability, whether it **prevents** the violation (kernel primitive or pre-tool hook), only **detects** it (portable postflight), or **cannot honor** it. A required control the runtime cannot enforce **fails the gate closed** – waive only explicitly with `--accept-unenforced`, written into the profile.

THE CLI PATH

- 1 `cvg doctor runtime-contract` attest the host first
- 2 `cvg bind --task <spec>` freeze profile + brief + adapters
- 3 `cvg bind --check --task <spec>` read-only freshness recheck
- 4 `cvg gate --path <p>` ask the repo fence about one path

NEVER UPGRADED

A profile never upgrades **detect** to **prevent** because the vendor has an optional hook. Declared assurance must match what the resolver **proves**.

● ● ● cvg/execution/T-20260730-add-health-endpoint/execution-profile.yaml ANATOMY OF THE CONTRACT

```
task.spec_ref: { path, sha256 } – the spec pinned by content, not name
write_scope: touches_paths + creates_paths · forbidden: do-not-touch
context: ADRs · approved knowledge · cached docs – each with an evidence hash
topology: single | single-explorer | implementer-verifier | parallel (static reason required)
policy.external_writes: deny – commit ≠ push ≠ tracker ≠ PR: four distinct effects
enforcement.assurance: weakest-link across controls
+ AGENTS.task.md – the brief pins IDENTIFIERS, never content:
"done is the Exit Check exiting zero – not your judgement."
```

THE FENCE IS CHECKED TWICE

per-candidate + whole-diff

Pre-write, per candidate path (vendor hook); and pre-settlement, over the whole diff – **mandatory even when** a stronger sandbox exists.

GATE REFUSALS

CHECK_RUNTIME_CONTRACT=FAIL

Unsigned task · stale spec hash · an XL/XXL node instead of a leaf · missing evidence hashes · a parallel topology without disjoint write ownership.

8 The Loop – attempt → verify → repeat, with brakes.

SKILL · task-loop

GATE · TASK_LOOP=SETTLED

LANES · ALL

IN · ONE ISSUE + BOUND CONTRACT

OUT · PR + RECEIPT, OR AN HONEST FAILURE

WHY IT'S A LOOP AND NOT A GATE

Each attempt is a fresh process – state lives on disk, never in a growing conversation (a retry otherwise re-reads every prior failure at quadratic cost). **Brakes are checked before each engine call**, where the money isn't spent yet. Budget flags can only **tighten**: a loop whose ceiling can be raised at the call site has no ceiling.

RED TWICE, SAME WAY → DIAGNOSE

Stop patching, start diagnosing: reproduce minimally → hypothesize in one sentence → **verify the hypothesis before writing the fix** → fix once, re-run. A fix applied to a confirmed cause converges in one iteration; a fix applied to a guess converges by accident.

THE CLI PATH

- 1 `cvv loop --estimate --issue <id>` read-only budget preflight
- 2 `cvv loop --issue <id> --agent codex` the bounded descent
- 3 `touch cvv/loop/<id>/STOP` cancel at the next brake check
- 4 `cvv loop --issue <id> --resume` continue from the checkpoint

NEVER `git add -A`

Settlement stages only the contract's `fs.write` scope – `git diff` never lists untracked files, so a blanket add could smuggle an out-of-scope file unseen. Every staged path is re-checked before commit.

cvv/loop/T-20260730-add-health-endpoint/

ANATOMY OF A RUN

`state.env` – the durable checkpoint: `ITER · STRIKES · TOKENS_USED · fingerprint`
`brief-003.md` – spec path · contract · attempt 3 of 15 · **only the failing checks**, plus "what the previous attempt did (do not repeat it)"
`attempts/003.log` – engine transcript + verify output, one file per attempt
`HANDOFF.md` – why it stopped · budgets used · last failure · how to resume
`STOP` – touch to cancel; honored at the next brake check, then deleted
 — settlement — `branch task/<id>-<slug> · scoped staging · receipt written LAST`
`.../cvv/STATE.md` – append-only board: | when | task | engine | state | iters |

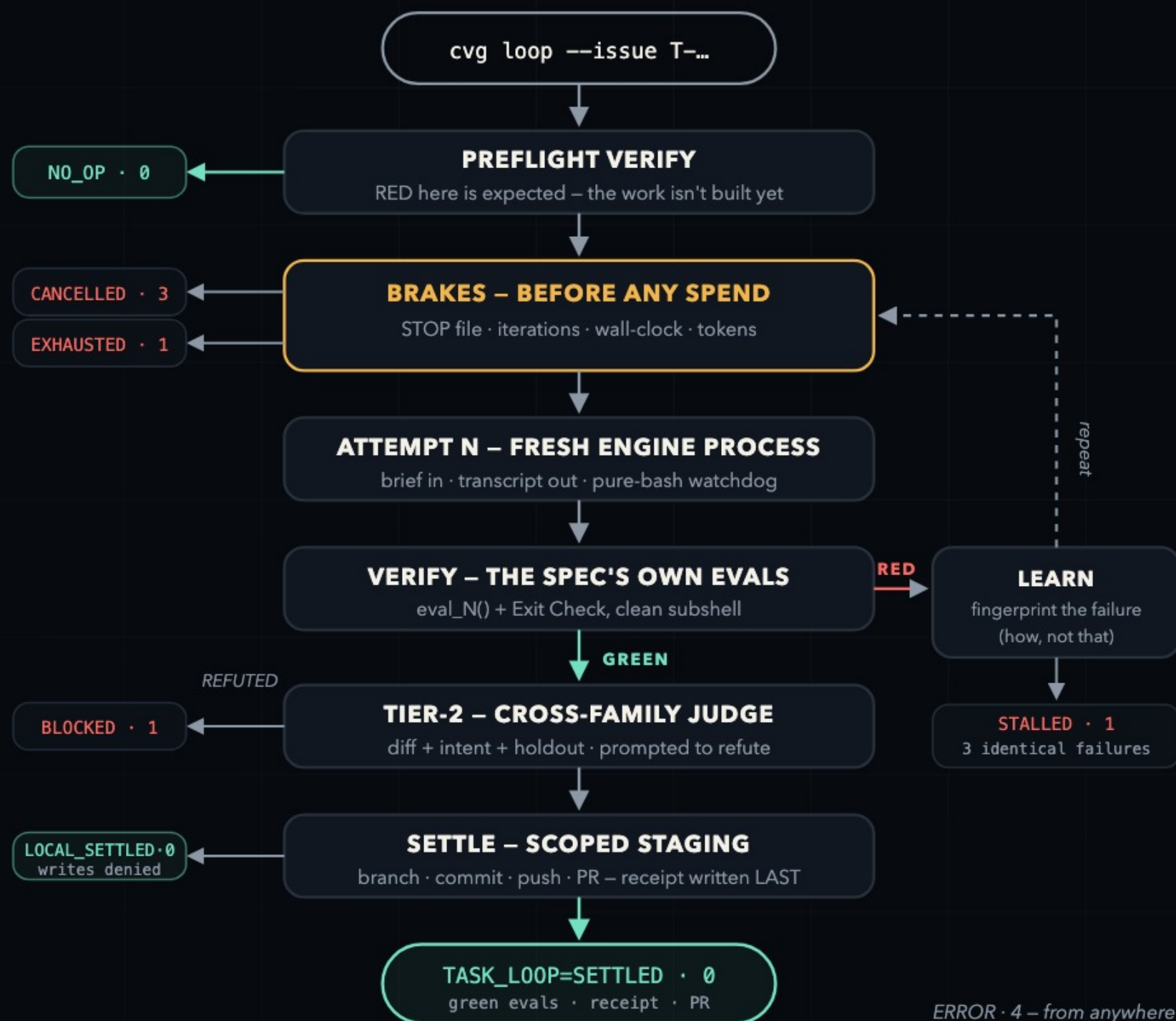
ENGINES ARE ONE FILE EACH

`engines/<name>.sh` – two calls
`--available` and `--prompt-file F --workdir D`. Adding an engine is a new file, never a kernel change. A pure-bash watchdog caps every attempt's wall-clock (exit 124).

UPSTREAM GAPS STAY UPSTREAM

`broken eval = Pass 5 fix`
 Broken eval bash, a missing ADR, a stale profile – each is a named upstream gap. The loop reports **BLOCKED**; it never hacks the eval.

Eight ways a loop can land. All of them named, only three exit zero.



THE EIGHT TERMINAL STATES

| | |
|--|---|
| SETTLED
green evals, external writes permitted, PR opened
exit 0 | LOCAL_SETTLED
green evals; policy denies external writes – local commit
exit 0 |
| NO_OP
already green on arrival – nothing to do, honestly said
exit 0 | BLOCKED
needs a human, an upstream input, or tier-2 REFUTED
exit 1 |
| STALLED
the same check failed the same way 3 times – believe it
exit 1 | EXHAUSTED
a budget ceiling hit – WIP committed, HANDOFF.md written
exit 1 |
| CANCELLED
the STOP file arrived; honored at the next brake check
exit 3 | ERROR
the loop itself could not continue safely
exit 4 |

THREE BUDGET AXES – FLAGS ONLY TIGHTEN

iterations budget_iterations, default 15 · **wall-clock** timeout_minutes, default 90 (accumulated working time, so a resumed run isn't pre-exhausted) · **tokens** budget_tokens, unenforced unless declared – absence is "unknown", never zero.

THE STAGNATION FINGERPRINT

sha256 over the [fail]/[pass] lines + the Exit Check line, **durations stripped** – the same failure taking 1s instead of 0s must not read as progress. Identical fingerprint = a strike; three strikes = **STALLED**.

The trust chain. From a 256-bit key to a receipt that can't lie.

● ● ● the seal, end to end

HMAC-SHA256 v2

- ① **provision** `cv g setup signing` — 256-bit key at `.git/info/`, mode 0600, never enters git
- ② **stamp** `cv g tasks gate --stamp` — seals the canonical payload:
`id · body_digest · authz_digest (paths, deps, effort, budgets) · signed_off fields`
- ③ **verify** recomputed at `validate · bind --check · accept · receipt --post-hoc`
- ④ **survive** Pass 6's `tracker_ref` stamp touches metadata only — the seal holds
- ⑤ **break** one eval byte changes → **DO NOT DELEGATE: modified after stamping**

THE THREE-RUNG TRUST LADDER

Tier 1 — key present, MAC verifies → full crypto trust, unsupervised crank OK · **Tier 2** — key or signature absent → structural-only, **supervised dispatch only** · **Tier 3** — MAC mismatch → hard fail. Honest threat model: symmetric HMAC is **tamper-evident, not tamper-proof** — it binds “a repo-key holder stamped this”, and per-author signatures (Ed25519/DSSE) are the named upgrade.

RECEIPTS — A HASH CHAIN, WRITTEN LAST

`cv g/receipts/<task-id>.json (cv g.execution-receipt.v1)` pins the spec sha256, the profile sha256, the eval output digest, agent, and branch. A superseded receipt is archived as `.attempt-<sha12-of-its-own-bytes>` — and the **pass** receipt is written last, so it can never attest to a settlement that didn't happen.

TIER-2 VERIFICATION — THE HOLDOUT

The spec may carry a **## Holdout** section: assertions **excluded from the worker's brief**, reserved for the judge — train/test separation borrowed from ML. `cv g verify` sends the diff, the stated intent, and the holdout to a different-family model prompted to refute. It never sees the worker's transcript.

BLAST RADIUS IS COMPUTED, NOT CLAIMED

High if any touched path matches `auth · session · token · secret · payment · billing · migration · schema`, or `effort ≥ L`, or the change spans `> 5` files. Low + **UNAVAILABLE** → recorded and permitted. High + **UNAVAILABLE** → **blocked**, with three exits: a second engine, a smaller change, or an explicit `--accept-unverified`.

THE FULL CHAIN, IN TOKENS

```
$ cv g tasks gate --stamp ... → VERDICT: DELEGATE · TIER=1
$ cv g bind --check --task ... → CHECK_RUNTIME_CONTRACT=PASS
$ cv g loop --issue ... → TASK_LOOP=SETTLED
$ cv g verify --task ... → CHECK_VERIFY=UPHELD
$ cv g register --check → CHECK_REGISTER=OK
```

— **The invariant:** policy is not part of the signed payload — re-signing a spec can never buy access the fence denies, and hand-stamping is rejected. The only path to autonomy is through the gate.

Proven in both directions. No Anthropic model in either seat.

● ● ● obs-rowcounts · 2026-07-29

CHECK_VERIFY=UPHELD

builder: codex (openai) – green in one iteration
judge: kimi (moonshot) – upheld every holdout assertion
outcome: TASK_LOOP=SETTLED – PR #9 merged, receipt on file

● ● ● obs-fence · 2026-07-29

CHECK_VERIFY=REFUTED

builder: kimi – built it, evals GREEN
judge: codex – found the fence swallows OSError and reports an unreadable surface as FENCE=OK – exactly what the holdout reserved for the judge
outcome: TASK_LOOP=BLOCKED – settlement refused, handoff written; left BLOCKED on the board deliberately, as the living demonstration

“A green eval is necessary, not sufficient – demonstrated, not asserted.”
THE REFUTED VERDICT CAUGHT A FAIL-OPEN BUG THE DETERMINISTIC EVALS COULD NOT SEE

| | | | | | |
|-----------------------------|--------------------|--------------------------|-------------------------|-------------------------|--------------------------------|
| 21 | 52 | 145 | 9/9 | 1 | 0 |
| HERMETIC CI SUITES · 2 OSes | LOOP-KERNEL CHECKS | REGISTER CHECKS, OFFLINE | BACKLOG SETTLED VIA PRs | FULLY UNATTENDED SETTLE | SECRETS OR LIVE SERVICES IN CI |

Provenance: Converge practices what it enforces – this repository's own backlog was driven through the loop (board parity 9⇌9, empty ready frontier), the clean-room suite proves init → sign → bind → RED→GREEN loop → acceptance → receipt hash chain from an empty repo, and a version-unity gate fails the build if any manifest drifts from **VERSION**.

Twelve portable skills. Nine spine, three utility – one install, every harness.

UTILITY · THE CONDUCTOR

evidence-to-next-pass **cvg next**

Derives where the descent stands from workspace evidence – no stored state. Prints the evidence board, the owning skill, its steering prompt, and the closing gate. Pre/post hooks enforce order fail-closed. `NEXT_PASS=0-8|DONE`

UTILITY · THE TEACHER

pass-to-lesson **cvg lesson**

After any pass closes: every component, the decision it encodes, **what breaks downstream without it**, plus check-yourself questions. -- **immutable** proves teaching changed nothing it taught. Optional – never blocks the descent. `CHECK_LESSON=PASS`

UTILITY · THE FORGE

skill-creator

Author and validate new skills. Its `quick_validate.py` is the bar every shipped skill passes – and the same check the installer runs before it reports `INSTALL=OK`.

THE SPINE – ONE SKILL PER PASS

0 · idea-to-brd

grill the idea into a signed BRD – or a no-go record

1 · brd-docs-to-tech-req

falsifiable requirements, stable R-n ids, typed gaps

2 · tech-req-to-adrs

ground in the running system; immutable, evidenced ADRs

3 · reqs-to-swimlane-plans

seams → lanes → legs; steel thread; H1-H5 hardening

4 · sketch-plans-adversarial-review

cross-family attack, FIX/ACCEPT, provenance-stamped

5 · task-spec

the cornerstone: evals first, six tiers, the HMAC seal

6 · task-specs-to-issues

1:1 board projection, blocked-by edges, six-verb adapters

7 · task-to-runtime-contract

epoch-scoped authority, evidence hashes, write fence

8 · task-loop

bounded attempt → verify → repeat; eight named landings

Portable by construction: skills are plain markdown + scripts in each harness's native discovery directory – `.claude/skills/` · `.agents/skills/` (Codex + Kimi) · `.grok/skills/`. Any harness that can run shell commands can drive the gates.

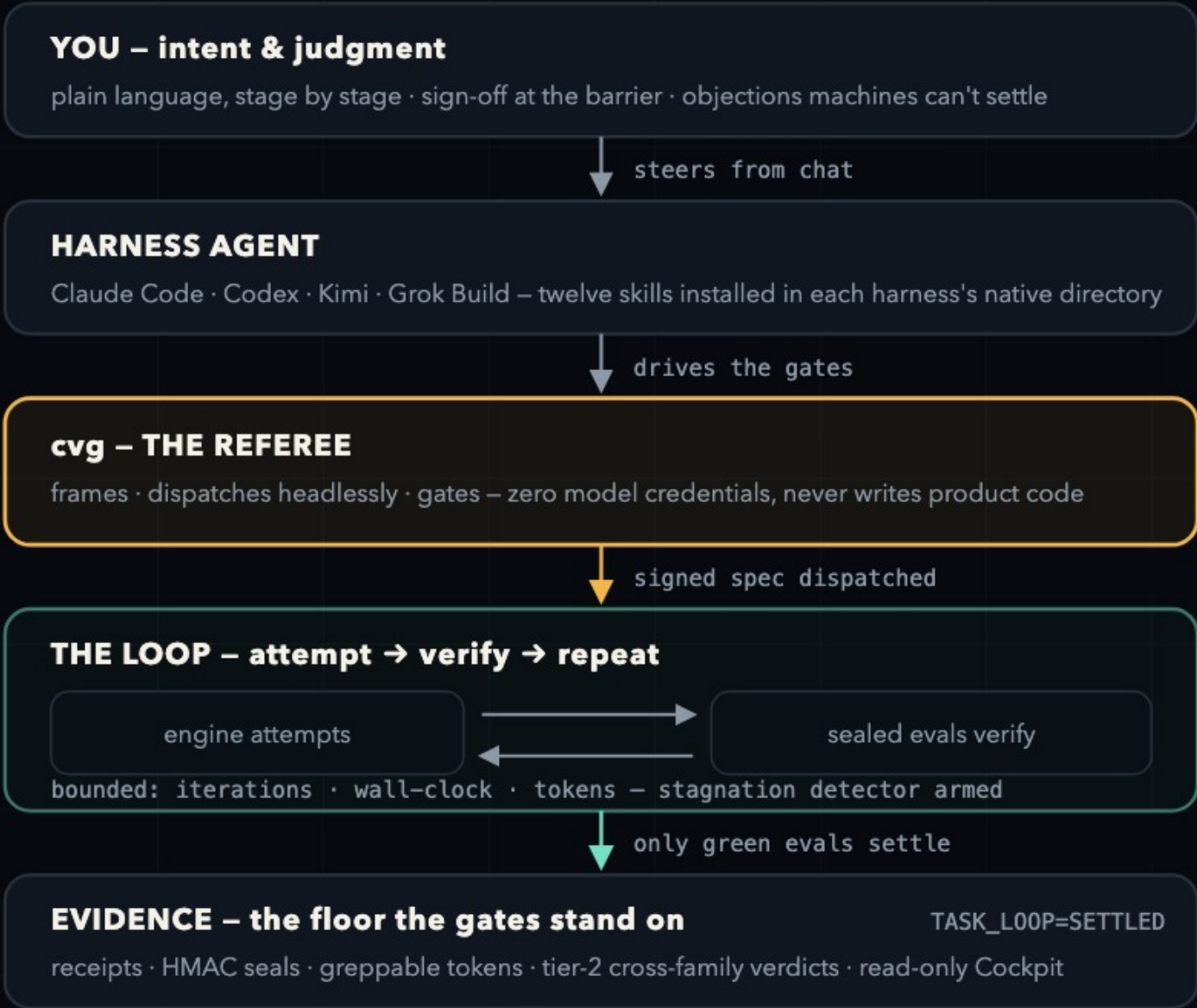
One referee. Every verdict ends in one greppable token.



WHAT A SESSION ACTUALLY LOOKS LIKE

```
$ cvg tasks gate --stamp cvg/tasks/T-20260730-add-health-endpoint.md
VERDICT: DELEGATE
TIER=1
$ cvg loop --issue T-20260730-add-health-endpoint --agent codex
TASK_LOOP=SETTLED # exit 0 – only SETTLED · LOCAL_SETTLED · NO_OP exit zero
```

THE TRUST ARCHITECTURE



- EXIT CODES ARE CONTRACTS – RETRYABLE IS PUBLISHED
- 0 OK – gate passed / command succeeded

1 GATE_FAIL – verdict RED; artifact not acceptable

2 USAGE_ERROR – bad flag or nothing to gate

20 SKIP – engine not installed *retryable*

21 TIMEOUT – wall-clock cap, fail-closed *retryable*

22 ENGINE_ERROR – no parseable judgment *retryable*

3 CANCELLED – the loop's STOP-file landing

4 ERROR – the loop could not continue safely

Agent-native by design. The whole surface, one JSON call.

cvrg agent-context – THE SELF-DESCRIPTION MANIFEST

```
{
  "schema_version": "1.0", "tool": "cvrg", "role": "referee",
  "contracts": { stdout, stderr, token, color, determinism, resolution },
  "global_flags": [ {flag, position, effect} ... ],
  "exit_codes": [ {code, name, meaning, retryable, side_effects} ... ],
  "commands": [ {name, converge_pass, wraps, emits_tokens,
                  mutating, example, description} ... ]
}
```

--json – THE UNIFORM RESPONSE ENVELOPE, ANY COMMAND

```
{ ok, command, token, verdict, exit_code, changed, dry_run,
  data, error, warnings, meta{schema_version, tool, cvrg_version} }

# a pure re-invoke wrapper – the byte-parity default path is untouched;
# the underlying exit code is preserved. Proven 14/14.
```

--dry-run – PREVIEW WITHOUT PERFORMING

Applies to mutating commands only (review --adversary, bind, tasks gate --stamp, transition, register, setup writes). Reports the intended action, changes nothing: changed=false · dry_run=true · DRY_RUN=OK. Read-only verification has its own doors: tasks validate --no-state, bind --check, register --check.

SETUP – CHOICES, NEVER CREDENTIALS

cvrg setup is a readiness board ending in SETUP=READY with your exact next step. setup key reads the tracker key from **stdin**, **never argv**, and stores it once outside the repo. setup harness scaffolds a ~50-line project router and stops – **cvrg never generates doctrine**.

WHY AGENTS DON'T MEMORIZE THE SEQUENCE

cvrg next names the current pass, its steering prompt, and its closing gate – derived from evidence. cvrg ready answers “what now?” between tasks – the dispatchable frontier only (ready **and** unblocked), failing closed on a dangling or cyclic edge.

The control plane. Two tracked files answer “may ANY task ever write here?”

... `.cvg/gate.yaml` – tracked, reviewed like any other file

THE REPO WRITE FENCE

```
version: 2
protected_paths: # the fence a Task-Spec cannot widen
  **/.git/** · **/.cvg/** · **/.env · **/secrets/** · **/credentials/**
  **/*_key* · **/*_secret* · **/id_rsa* · **/.terraform/**
  **/migrations/** · **/auth/** · **/payments/** · **/billing/**
  .cvg/gate.yaml · bin/cvg · skills/task-*/scripts/** · .github/workflows/**
  # "an agent that edits its own verifier is the specification-gaming failure mode"
max_changed_files: 12 # blast radius, orthogonal to paths
```

ASK THE FENCE – READ-ONLY, ANY TIME

```
$ cvg gate --path src/api/health.py
  allowed src/api/health.py
$ cvg gate --path auth/tokens.py
  REFUSED auth/tokens.py (rule: **/auth/**)
CHECK_GATE=OK|ABSENT|FORBIDDEN|ERROR
```

PRECEDENCE – THE FENCE ALWAYS WINS

The Pass 7 contract answers “may **this** task write here?”; the fence answers “may **any** task ever write here?”. **Protected paths beat contract, always** – and policy is not part of the signed payload, so re-signing a spec cannot buy access.

THE WORKSPACE TREE

`cvg/` is the factory floor: `docs/{brd,tech-spec,no-go,adrs,lessons} · swimlanes/ · tasks/ · execution/ · loop/ · receipts/ · brain/`. Paths resolve against the **workspace, never the git root** – `cvg/` may live in a subproject.

WHAT LIVES WHERE

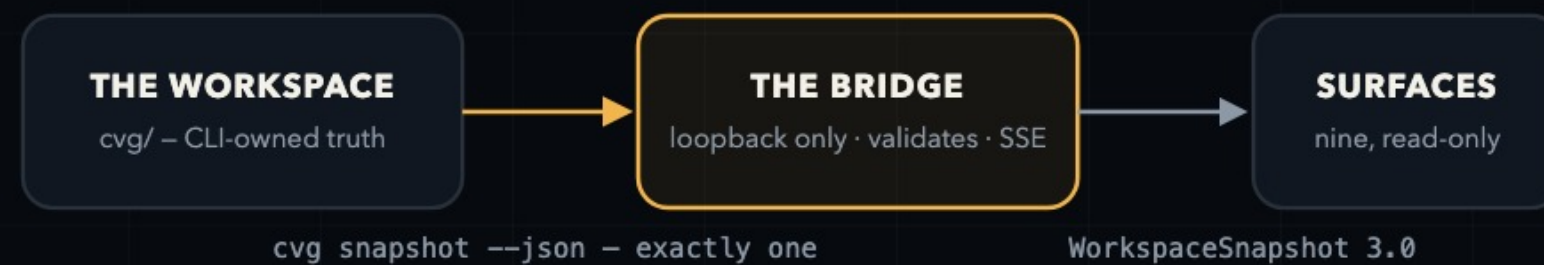
`.cvg/gate.yaml` – tracked fence · `.cvg/config` – choices, never credentials · `.cvg/identity` – tracker identity map, mode 0600 · `.git/info/taskspec-signing-key` – the HMAC key, 0600, **never enters git** · tracker API keys – the OS keychain.

ENGINES AUTHENTICATE THEMSELVES

The referee holds zero model credentials. Engine CLIs (`c`laude, `c`odex, `k`imi, `g`rok) are needed only by the passes that dispatch – each brings its own auth.

Cockpit. Observable, without a second source of truth.

A repository-first frontend that renders the real nine-pass lineage, work queue, runs, evidence, receipts, health, and adversarial objections. **Deliberately read-only** – the CLI still owns execution and proof.



SHA-256-BOUND PREVIEWS

an artifact preview requires the snapshot ID + captured SHA-256; changed bytes → HTTP 409 – changed bytes can never masquerade as captured proof

WHAT IT CANNOT DO – BY CONSTRUCTION

never executes Converge work: cannot create, run, approve, register, bind, transition, or settle; permission requests are always rejected; a replay is never labeled live

NINE SURFACES

Ask · Overview · Journey · Decompose · Work · Runs · Docs · Activity · Health – the descent's lineage, gate results, the queue, execution attempts, readable documents, objections and blocking questions, receipts, and the command `cvf` currently authorizes.

FOUR FACTS, KEPT SEPARATE

A deterministic gate verdict · an adversarial review verdict · the conductor's current authorization · whether an artifact still matches the snapshot digest. *"A matching hash proves which bytes were observed, not that their claims are current or approved."*

RUN IT

```

$ npm run cockpit:install
$ npm run cockpit:dev -- --cvf-home "$PWD" \
  --project-root /path/to/workspace
→ http://127.0.0.1:4173
  
```


You steer in plain language. The agent drives the gates.

you I want a storm-alerts feed for our customers. Start a Converge run – capture it.

agent Pass 0 (idea-to-brd): drafting the BRD, grilling the gaps out of the idea... **CHECK_BRD=PASS** – the brief is signed.

you Take it to requirements and architecture.

agent Pass 1: **CHECK_TECH_SPEC=PASS**. Pass 2: five ADRs recorded – **CHECK_ADR=OK**.

you Break it down, then have Codex attack the plan.

agent Pass 3: two swimlanes, six legs – **CHECK_PLAN=OK**. Pass 4: codex raised 3 objections. Two resolved; one needs your call.

you Objection two is right – merge the retry logic into one leg. Then sign.

agent Plan amended. **CHECK_CONSENSUS=OK** – the barrier is behind us.

you Cut the tasks, put them on the board, and ship the first ready one.

agent Pass 5: six Task-Specs, evals first, all **TIER=1** and sealed. Pass 6: **CHECK_REGISTER=OK**. Pass 7: **CHECK_RUNTIME_CONTRACT=PASS**. Pass 8: attempt → verify → repeat... green on iteration 2. **TASK_LOOP=SETTLED** – PR opened, receipt written. Next ready task: 2 of 6.

THE DIVISION OF LABOR

You own intent and judgment; the factory owns procedure and proof. Your voice is needed at exactly the moments that deserve it: the barrier sign-off after the adversarial review, and any objection the machines cannot settle. Everything else runs on gates.

WHY THIS WORKS

The skills are installed **in your harness**, and **cvg agent-context** hands any agent the whole surface as one JSON manifest. You never memorize the sequence – **cvg next** derives it, **cvg lane** routes it, **cvg setup** names the exact missing step, **cvg ready** answers “what now?”.

SWAP THE ENGINE, KEEP THE GATES

—**agent claude** → **codex** → **kimi**: the same signed spec dispatches to any of them, and the gate that scores the work is identical. Engines are one adapter file each.

Three doors, same result. Twelve skills, every harness, cvg on PATH.

DOOR ① · PLUGIN MARKETPLACE

Inside Claude Code

Grok Build reads Claude marketplaces natively. Ships the skills **and** puts **cvg** on the session PATH – nothing to authenticate.

```
/plugin marketplace add luanmorenommaciel/converge
/plugin install converge@cvg
```

DOOR ② · npm / npx

Anywhere Node lives

The package embeds the CLI **and** all skills. Project-local works too: **npm i -D** then **npx cvg**

```
npm install -g github:luanmorenommaciel/converge
cvg-install
```

DOOR ③ · ONE-LINE SHELL

No Node – git + bash only

Pin a release with **CVG_REF=v0.1.0**; install from a fork with **CVG_REPO_URL=<url>**.

```
curl -fsSL https://raw.githubusercontent.com/luanmorenommaciel/converge/main/install.sh | bash
```

WHERE SKILLS LAND – ONE INSTALL COVERS ALL FOUR

| | | |
|--------------|-----------------|-------------------------------|
| Claude Code | .claude/skills/ | + marketplace plugin |
| Codex + Kimi | .agents/skills/ | the AGENTS.md family |
| Grok Build | .grok/skills/ | reads Claude marketplaces too |

GUARANTEES

Idempotent – re-running skips existing entries (**--force** to replace). **Never writes a credential.** Verifies the installed skills parse before reporting **INSTALL=OK**. Development: **install.sh --symlink** keeps skills live-linked to your checkout.

- REQUIRES · BASH 3.2+ AND GIT
- GATES · PYTHON3 STDLIB, NO PIP
- NODE · NPM DOOR ONLY
- ENGINES · AUTHENTICATE THEMSELVES
- WINDOWS · GIT BASH / WSL

— **First minutes in a repo:** `cvg init` → `cvg setup signing` → `cvg setup` – then `cvg lane "your next change"`. Init is non-clobbering; nothing forces the full descent.

Done is proven.

Converge 0.1.0 ships the CLI, twelve skills, and the Task-Spec engine as **one versioned unit** – the first release where every claim has a receipt behind it. One number everywhere: **VERSION** is the single source of truth, and a CI gate fails the build on drift.

DELIBERATELY NOT IN 0.1.0 – THE ROADMAP

The Manager – fleet dispatch across ready tasks (the loop is single-task by design today). Git-native orchestration: Actions as scheduler, the PR as settlement, branch protection as the gate. Top of the roadmap. · **The CI eval-gate** – server-side re-verification of every green claim.

v0.1.0 · 2026-07-30

MIT – ONE LICENSE, WHOLE UNIT

CI GREEN · macOS + LINUX

BUILT THE WAY IT ASKS YOU TO BUILD

This repository's own backlog went through the loop – nine tasks settled through merged PRs, one fully unattended, board parity 9⇌9. Built with Claude Code, Codex, and Kimi in all three seats: author, adversary, and judge.

READ NEXT

| | |
|-------------------------|---------------------------|
| docs/task-spec-v0.1.pdf | the cornerstone, in depth |
| skills/README.md | the skill catalog |
| bin/README.md | the CLI surface ledger |
| presentation/*.html | interactive walkthroughs |

```
# the whole method, one breath:
capture → intent → structure → decompose
→ consensus (the barrier) → tasking (the seal)
→ register → bind → loop until SETTLED
# evidence feeds the next pass. crank → learn → update.
```



COMPILE INTENT INTO SHIPPED SOFTWARE